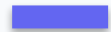


Design Patterns

Named, proven solutions to problems you'll meet again and again.

Patterns Put SOLID to Work

A design pattern is a named, reusable solution to a recurring design problem. It's shared vocabulary — and SOLID principles made concrete.



Factory

Create objects without coupling to their concrete types.



Strategy

Swap an algorithm at runtime behind one interface.



Proxy

Stand in for an object to control access to it.

Learning Objectives

- Recognize the problem each of the three patterns solves.
- Implement Factory, Strategy and Proxy in a clean, minimal way.
- Connect each pattern back to SOLID (especially OCP and DIP).
- Know when not to use a pattern — patterns are tools, not trophies.

A Vocabulary — and a Trap

Patterns give you

- Shared language: 'use a Strategy here'
- Proven structures, fewer surprises
- SOLID applied, not just preached

But beware

- Pattern for its own sake = complexity
- Simplest thing first; pattern when it pays
- A name is not a justification

Pattern 01

Factory

Create objects without hard-wiring their concrete types.

The Problem With 'new'

Calling `new SomeClass()` everywhere welds your code to that concrete type. Change the type, and you edit every call site.

- Scattered new couples callers to concretions (violates DIP).
- Adding a new variant means editing existing code (violates OCP).
- A factory centralises creation behind one seam.

Centralize Creation

Java

One place knows the concrete types

```
interface Notifier { void send(String msg); }

class NotifierFactory {
    static Notifier create(Channel c) {
        return switch (c) {
            case EMAIL -> new EmailNotifier();
            case SMS    -> new SmsNotifier();
        };
    }
}

// callers ask for a Notifier; they never say 'new EmailNotifier'
```

Why It Helps



Before

- new EmailNotifier() sprinkled everywhere
- New channel → edit many files
- Hard to test — real objects only



After

- Callers depend on the Notifier interface
- New channel → add one case in the factory
- Inject a fake Notifier in tests

Pattern 02

Strategy

Select an algorithm at runtime — behind one stable interface.

The Growing if/else

When behavior varies by a 'kind', an if/else or switch tends to grow forever — and every new kind edits the same method.

- Mixed algorithms in one method = low cohesion, hard to test.
- Each new case edits working code (violates OCP).
- Strategy makes each algorithm its own swappable object.

Each Algorithm, Its Own Object

Tangled

```
double fee(String kind, double amt){  
    if (kind.equals("card")) return amt*0.02;  
    if (kind.equals("wire")) return 5.0;  
    // new kind? edit here again  
}
```

Strategy

```
interface Fee { double of(double amt); }  
class CardFee implements Fee {...}  
class WireFee implements Fee {...}  
double fee(Fee f, double a){ return f.of(a); }  
// new kind = new class
```

Strategy is Open/Closed in action — and the same shape as OCP from last session.

Where You've Seen It

- Sorting with a comparator, pricing rules, retry/back-off policies.
- Payment methods, compression algorithms, validation rules.
- Anywhere you'd otherwise write 'if type == ...'.

Pattern 03

Proxy

A stand-in that controls access to the real object.

Add Behavior Around an Object

Sometimes you need to do something before or after using an object — cache, log, check access, or defer creation — without changing the object itself.



Proxy flavors

- Virtual — create the real thing lazily
- Protection — enforce access rights
- Caching / logging — wrap each call



The key idea

- Same interface as the real object
- Callers can't tell the difference
- Extra behavior lives in the proxy

Same Interface, Extra Control

Java

The proxy defers the expensive load

```
interface Image { void render(); }

class RealImage implements Image {           // heavy: loads from disk
    RealImage(String path) { load(path); }
    public void render() {...}
}

class LazyImage implements Image {           // virtual proxy
    private RealImage real; private final String path;
    public void render() {
        if (real == null) real = new RealImage(path); // load on first use
        real.render();
    }
}
```

Where You've Seen It

- ORM lazy loading, Spring/AOP proxies, caching layers.
- Access control and rate limiting in front of a service.
- Logging, metrics and retries wrapped transparently around calls.

Patterns Are SOLID, Named

Factory

Serves DIP — callers depend on an interface, not a concrete type.

Strategy

Serves OCP — add behavior by adding a class, not editing one.

Proxy

Serves OCP & SRP — add cross-cutting behavior without touching the core.

Patterns as Prompts

Pattern names are dense, precise instructions an AI understands well.

- 'Refactor this switch into a Strategy' communicates intent in three words.
- Ask the AI which pattern fits — then judge whether it's worth the complexity.
- Resist gold-plating: the simplest design that works still wins.

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 3 tests here

patterns/factory/
NotifierFactory hides the concrete types

patterns/strategy/
FeeCalculatorSmell → Fee strategies

patterns/proxy/
LazyImage defers the real load

C# / .NET

xUnit · 3 tests here

Patterns.cs
Factory · Strategy · Proxy

PatternsTests.cs
one test per pattern

Python

pytest · 3 tests here

patterns.py
Factory · Strategy · Proxy

tests/test_patterns.py
one test per pattern

github.com/kaldiruglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

Each pattern keeps its smell. Read the pair before the pattern name — the shape is easier to remember than the label.

*Clone one, run the suite, then break something and watch which test goes red. **Across all chapters the suites are 70 · 63 · 65 tests.***

Remember these

Key Takeaways

- ✓ Patterns are named, proven solutions — and shared vocabulary.
- ✓ Factory: create objects without coupling to concrete types.
- ✓ Strategy: swap algorithms behind one interface (OCP in action).
- ✓ Proxy: a same-interface stand-in that controls access to the real object.
- ✓ Use a pattern when it pays — never for its own sake.

Go deeper

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Design Patterns: Elements of Reusable Object-Oriented Software** Gamma, Helm, Johnson & Vlissides · Addison-Wesley, 1994
The Gang of Four — the source for Factory, Strategy and Proxy as we teach them.

📖 **Head First Design Patterns, 2nd ed.** Freeman & Robson · O'Reilly, 2020
The same patterns, taught visually — the gentlest on-ramp for the room.

📖 **Refactoring to Patterns** Joshua Kerievsky · Addison-Wesley, 2004
*How to *arrive* at a pattern from smelly code, instead of designing it in up front.*

📖 **Patterns of Enterprise Application Architecture** Martin Fowler · Addison-Wesley, 2002
The next layer up: patterns for real applications, not just objects.

Online — pattern catalog with diagrams and code in 9 languages → refactoring.guru/design-patterns

What's Next

Module 6

Secure Coding

with Akin Kaldıroğlu

Input validation, safe error handling and resource management — writing code that an attacker can't turn against you.

Spot the Pattern That Fits

Find where a pattern removes a smell in the ACME Orders code — don't force one where it doesn't pay.

Refactoring course · Patterns in the Wild

Repo github.com/kaldiroglu/refactoring-course-student-materials

IN THIS SESSION, LOOK AT

- ▶ `.../notify/EmailSender.java` , `SmsSender.java` (Strategy)
- ▶ `.../report/ReportBuilder.java` (Factory / Builder)
- ▶ `.../service/ShippingCalculator.java`

Questions?

The right pattern, at the right time.